

The polilabs knowledge graph, from first principles

How it evolved, how every layer works, and why — no condensation

For Andrew — written June 12, 2026

Contents

I	Primitives: the concepts everything else uses	1
1	The problem statement, precisely	1
2	Property graphs vs. relational tables — and why polilabs uses both	1
3	The three retrieval primitives: FTS, embeddings, alias tables	2
4	Why round trips, not milliseconds, are the latency budget	2
5	Outcome derivation as event sourcing	3
6	The bipartisanship metric, with worked numbers	3
7	Coverage as a contract (the under/overcoverage primitive)	4
II	The data: where every fact comes from	4
8	The government data landscape	4
9	Identity: the two bill-ID schemes and the vehicle-bill trap	5
10	The two data layers: universe vs. curated topics	5
III	The storage architecture, layer by layer	6
11	Files are the source of truth	6
12	The SQLite index: every table and its job	6
13	The Kùzu graph: nodes, edges, and what is actually populated	7
14	Killing the 3.5-hour rebuild: the incremental path	8
IV	The evolution: how we got here	8
15	The chain at a glance	8
16	Why each step happened (the motivating failures)	9

V	The query surface: all 18 tools	9
17	The tool table	9
18	Routing: how the agent picks its first tool	10
VI	The code, walked	10
19	Primer: the two query languages	10
20	Outcome derivation (<code>ingest/billstatus.py</code>)	11
21	The alias contract (<code>index/build_universe_tables.py</code> + <code>api/_impl.py</code>)	11
22	The bill-level bipartisan number (<code>index/build.py</code>)	12
23	The card assembly (<code>api/_impl.py::get_bill_card</code>)	12
24	The universe build (<code>scripts/build_universe.py</code>)	13
VII	Cheat sheet: your questions, answered	13
25	Index of questions	13

This document builds the polilabs knowledge representation up from nothing: what problem it solves, the primitives it is made of, every data source it draws on, every storage layer, the full evolution from the v0.1 schema sketch to the current two-layer architecture, every agent-facing tool, and walks through the load-bearing code. Part VII is a cheat sheet indexed by the questions you have actually asked. Read it linearly the first time; use Part VII afterward.

Part I

Primitives: the concepts everything else uses

1 The problem statement, precisely

polilabs exists because of one empirical fact: commercial legal-AI tools hallucinate on 17–34% of queries (Magesh et al., Stanford RegLab 2024), and they do so not because LLMs are hopeless but because the substrate they are given — a keyword search box over editorial documents — is hostile to programmatic reasoning. The polilabs wager is that if you redesign the *substrate* (typed entities, verbatim text, explicit provenance, honest gaps), a large fraction of hallucinations collapse into either correct answers or honest “the corpus does not contain this.”

Everything in this document is downstream of three design commitments:

1. **Verbatim or nothing.** The system never paraphrases law before the user sees it. Extraction is mechanical (XML parsing, regex over action strings), and anything inferred carries provenance and confidence.

2. **Honest unknowns are a return type.** “Not in the corpus,” “no roll call exists,” and “status-only record” are first-class answers, not failures.
3. **The agent’s latency budget is round trips, not milliseconds.** See §4.

2 Property graphs vs. relational tables — and why polilabs uses both

A *relational table* stores rows with fixed columns; you ask questions by joining tables on key equality. A *property graph* stores typed nodes and typed directed edges, each carrying key-value properties; you ask questions by traversing edges (“from this Bill, follow `HAS_VERSION` then `HAS_SECTION`”).

The folk claim is that graphs are “better for connected data.” The honest version: graphs win when the question shape is a *path of unknown or variable length* (“what does this section cite, and what cites that?”), and relations win when the question shape is a *filtered scan or aggregate* (“count bills by outcome”). polilabs therefore runs both, over the same source files:

	SQLite (<code>data/polilabs.db</code>)	Kùzu (<code>data/polilabs.kuzu</code>)
Question shape	filter / aggregate / full-text	traversal / typed neighborhood
Examples	counts, outcome sets, name lookup	citation graph, defined-term scoping
Latency	~1–20 ms	~10–100 ms
Rebuild	minutes	hours (see §14)

Neither database is the source of truth. The *files* are (§11); both stores are disposable indexes rebuilt from them. This is the single most important architectural fact about polilabs: you can delete both databases at any time and lose nothing.

3 The three retrieval primitives: FTS, embeddings, alias tables

There are exactly three ways polilabs turns a user’s words into rows, and they have sharply different jobs.

Full-text search (FTS5 / BM25). SQLite’s FTS5 builds an inverted index: for every word (after *Porter stemming*, which maps “regulating”/“regulates”/“regulation” to one stem), a list of the rows containing it. BM25 ranks hits by term rarity and document length. This is the workhorse for topical queries (“facial recognition in schools”) and costs ~1–10 ms.

Dense embeddings. Each section of bill text is mapped by a small neural model (bge-small-en-v1.5, 384 dimensions) to a vector; a query is mapped to the same space; cosine similarity finds passages that are *semantically* close even with zero word overlap (“self-driving cars” → “autonomous vehicles”). polilabs fuses BM25 and dense rankings with reciprocal-rank fusion (RRF). Embeddings are the only layer with partial coverage right now (the embed pass is handed off; the coverage report says exactly how partial — §7).

Alias tables. A user who says “the CHIPS Act” is not doing topical search — they are naming an entity. The correct primitive is a *dictionary*: a table mapping every normalized known name to a canonical ID. The entity-linking literature is unambiguous that dictionary lookup comes first and covers most mentions (Wikidata alias tables alone resolve >50% of mentions before any ML), with fuzzy/embedding matching only as a fallback ranker. polilabs follows exactly this ladder (§21).

The design rule that falls out: *name-shaped queries hit the alias table, topic-shaped queries hit FTS(+dense), and the agent’s system prompt routes between them* (§18).

4 Why round trips, not milliseconds, are the latency budget

Every query primitive above answers in milliseconds. But the agent consuming them is an LLM in a tool loop: each tool call costs a full model generation–call–read cycle, roughly 1–3 *seconds* of model latency. A question answered with five sequential tool calls costs 5× that — 10–15 seconds — regardless of how fast the database is. So:

The unit of optimization is the number of LLM↔tool round trips, not per-query database time. A primitive that returns a complete, denormalized answer in one call beats five elegant fine-grained calls, always.

This is also the consensus of every deployed system we reviewed in the June 2026 research pass: Microsoft GraphRAG precomputes per-community summary reports so query time is “fetch a summary”; LightRAG’s headline result is *one* retrieval call per query versus hundreds for naive graph search; Apple’s Saga serves precomputed entity cards rather than traversing live. polilabs’ versions of this move are the aggregate primitives (`find_bills_defining`, `count_bills`, `find_universe_bills`) and the one-call bill card (§23).

5 Outcome derivation as event sourcing

A bill’s “outcome” (did it pass?) is not a field anywhere in government data. What exists is an *action trail*: dated strings like “Passed/agreed to in House” or “Cloture on the motion to proceed not invoked.” polilabs derives the outcome by scanning the trail for typed events and keeping the evidence:

actions $\xrightarrow{\text{regex probes}}$ events = [(type, date, verbatim evidence)] $\xrightarrow{\text{precedence rules}}$ outcome enum

The enum has 13 values: `enacted`, `vetoed`, `failed_passage`, `failed_cloture`, `passed_house_only`, `passed_senate_only`, `passed_both`, `died_after_passing_{house,senate,both}`, `reported_no_floor_vote`, `died_in_committee`, `pending`. The events list always travels with the label so an agent asked “why did it fail?” quotes dated evidence instead of inventing a narrative.

Worked example (the bug that scale found). H.R. 9495 (118th):

1. 2024-11-12 — “Failed of passage ... On motion to suspend the rules and pass the bill”
→ event `failed_passage`. (Suspension requires a 2/3 majority; the bill got 256–145, a majority but not 2/3.)
2. 2024-11-21 — “On passage Passed by the Yeas and Nays: 219–184” → event `passed_house`.
3. Congress closed with no Senate action.

A naive precedence (“any failure beats any passage”) labels this `failed_passage` — wrong; the bill passed the House nine days after the failed shortcut. The fix compares *last-event dates*: a dated passage supersedes an earlier failed floor vote (§20). The correct label is `died_after_passing_house`. None of the 232 curated bills exercised this path; bill #46,000-ish of the universe did. Scale is a test suite.

6 The bipartisanship metric, with worked numbers

For any roll call with party splits, define for each major party $p \in \{D, R\}$ the yea-share among that party’s yes+no votes, $s_p = \text{yea}_p / (\text{yea}_p + \text{nay}_p)$, and

$$\text{bipartisan_support} = \min(s_D, s_R) \in [0, 1].$$

1.0 means both parties unanimous yes; 0.0 means at least one party unanimous no. Display labels are thresholds on this number: `party_line` < 0.10 ≤ `cross_party` < 0.50 ≤ `bipartisan` < 0.90 ≤ `near_unanimous`.

Worked example — the two lives of H.R. 4346. The same bill number carried two very different coalitions:

- House roll 239 (2021, as the America COMPETES shell): 215–207. Democrats $s_D = 215/215 + \dots \approx 1.0$; Republicans essentially zero yeas, $s_R \approx 0.005$. `min` ≈ 0.005: `party_line`.
- House roll 404 (2022-07-28, final CHIPS concurrence): 243–187. $s_D = 219/219 = 1.0$; $s_R = 24/(24+187) = 24/211 = 0.114$. `min` = 0.114: `cross_party`.
- Senate roll 271 (2022-07-27): 64–33, $s_R = 0.347$: `cross_party` verging on bipartisan.

Two lessons. First, *the metric is per-vote*; a bill-level number must choose a vote, and polilabs uses the *final passage-class vote* (§22) because vehicle bills carry early votes from a prior life under another name. Second, the labels are display sugar — the number is what you filter and sort on, and you should quote the underlying splits in prose.

Why `min` and not, say, the share of the minority party? Because `min` is symmetric (it does not care who holds the majority) and it operationalizes “did *both* parties support this” — the secret-congress question — rather than “did the losing side defect.”

7 Coverage as a contract (the under/overcoverage primitive)

Agents fail on coverage in two symmetric ways. *Undercoverage error*: the agent believes the corpus knows less than it does and abstains wrongly. *Overcoverage error*: the agent believes the corpus knows more than it does and asserts from a facet that is only partially populated — the worse failure, because it reads as hallucination.

The fix is to make coverage a queryable, live-computed table rather than prose: for each topic and for the universe, for each data *facet* (actions, cosponsors, votes, outcome, CRS summary, embeddings, full text), report `bills_with_facet` / `bill_count`. Live-computed means it cannot drift from the index. The agent’s honesty rules become mechanical: before aggregating over a facet, check that the facet is total; when it is not, say which slice the answer rests on. This generalizes the schema’s original move for definitions (explicit gap nodes instead of silent misses) to the corpus itself.

Part II

The data: where every fact comes from

8 The government data landscape

Everything polilabs knows comes from five public, unauthenticated, primary sources. No aggregator (and no API key) sits in between — this matters because the two civic aggregators researchers used to rely on are dead (ProPublica Congress API archived 2025; GovTrack bulk data ended 2024).

Source	What it carries	How polilabs uses it
--------	-----------------	----------------------

GovInfo bulk XML	BILLS	Full bill text at each procedural stage (ih introduced, eh engrossed House, enr enrolled, ...), in USLM or pre-USLM XML	The curated corpora’s bill.xml ; parsed into the section tree; the most-final available version wins
GovInfo XML	BILL-STATUS bulk	Per-bill status: every title variant, structured sponsor/cosponsors (party, state, district), the complete dated action trail, recorded-vote pointers, public-law numbers, policy area, subjects	The universe layer (one zip per congress×type, 12 zips total); billstatus.json enrichment for curated bills; outcome derivation; the alias table’s main source
GovInfo bulk XML	BILL-SUM	CRS-written summaries, revised at each major action; the latest version describes the bill’s final state	universe_summaries table; the FTS summary column (topical search over all 46k bills); the card’s summary field
House Clerk / Senate LIS XML	roll-call	Per-vote: question, result, per-party totals, every member’s position with party and state	Party splits and bipartisan.support on every recorded vote; member-level positions for passage-class votes on curated bills
OLRC	Popular Name Table	The official act-nickname dictionary (popular name → public law)	926 alias rows for the covered Congresses, source-tagged olrc_popular_names

Update cadence note: BILLSTATUS bulk files are regenerated continuously by GPO (the directory listing showed same-day timestamps when we pulled); BILLSUM lags actions by days-to-weeks because humans at CRS write the summaries. This is why summary coverage is near-total for the closed 117th (15,221 of 15,231 bills) but partial for the 118th (9,399) and the in-progress 119th (4,256) — the facet table reports these numbers rather than hiding them.

9 Identity: the two bill-ID schemes and the vehicle-bill trap

Every bill has two equivalent IDs: the legacy form **117-hr-4346** (SQLite, file paths, tool arguments) and the URN form **bill:us/117/hr/4346** (the graph, where cross-jurisdiction extensions need a namespace). Section IDs are **{bill_id}::{xml_id}**, where **xml_id** is the USLM id attribute — globally unique within a bill version, which is what lets graph rows be scoped to a bill by string prefix.

Two identity facts that will bite anyone who does not know them:

1. **Bill numbers do not persist across Congresses.** “Laken Riley Act” names 118-hr-7511, 119-hr-29, and 119-s-5; only the last was enacted. This is why **lookup_bill** returns *all* matches with **is_ambiguous=True** and the agent is instructed to disambiguate, never to silently pick.
2. **Vehicle bills.** Congress amends a convenient existing bill rather than starting fresh, so a bill’s roll calls can belong to a prior life (H.R. 4346’s 2021 votes are America COMPETES votes), and a famous vote can live on a *different* number (KOSA’s 91–3 Senate vote was on the S. 2073 vehicle; S. 1409 itself correctly shows no floor vote). The data layer handles this by dating every vote and using final-passage votes for bill-level metrics; the prompt layer instructs the agent to check dates and questions before attributing margins.

10 The two data layers: universe vs. curated topics

This is the central organizing idea after the June 2026 expansion:

	Universe layer	Curated topic corpora
Population	every law-track bill (hr, s, hjres, sjres) of the 117th–119th: 46,168 bills	hand-curated or criteria-gated subsets: 232 bills in 3 topics
Carries	status: outcome + evidence, every known name, sponsor party/state, cosponsor party counts, roll-call party splits, CRS summary, policy area	everything the universe has <i>plus</i> full text: 167k sections, defined terms, amendment operations, citations, member-level vote positions
Source artifact	<code>data/universe/*.jsonl.gz</code> (committed, ~15 MB)	<code>data/corpus/{topic}/{bill}/</code> directories (committed)
Answers	“did X pass? was it bipartisan? what is it about? how many...”	“what does Sec. 3(b) say? how does it define ‘foundation model’? what does it amend?”

The boundary is stated as data (`in_corpus` on every universe row, the `full_text` facet in the coverage report) and enforced in prompts: a status-only card tells the agent *not* to call text tools on that bill. Topics so far: `ai_governance` (191 bills, criteria-gated: `corpus/inclusion_criteria.md`), `redistricting` (12, hand-curated seed), `secret_congress` (29, hand-curated passage-dynamics exemplars in six clusters: `corpus/secret_congress_criteria.md`).

Part III

The storage architecture, layer by layer

11 Files are the source of truth

Layer	Location	Contents	Disposable?
0. Cache	<code>data/cache/</code>	raw zips, BILLSTATUS XML, roll-call XML	yes (gitignored)
1. Truth	<code>data/corpus/</code> , <code>data/universe/</code>	per-bill dirs; compact jsonl.gz	no (committed)
2. Index	<code>data/polilabs.db</code>	SQLite: relational + FTS + embeddings	yes (rebuilt)
3. Graph	<code>data/polilabs.kuzu</code>	Kùzu property graph	yes (rebuilt)

A curated bill directory contains: `bill.xml` (verbatim text), `metadata.json` (identity, title, sponsor, topic, tier, and — for `secret_congress` — `cluster` and `curator_note`), `billstatus.json` (the keyless enrichment: outcome, events, votes with members, actions, cosponsors), `provenance.json` (source URLs, fetch times, criteria version). The rule “never modify `metadata.json` after promotion; enrichment goes in sibling files” is what made it safe to backfill all 203 pre-existing bills without touching anything the original pipeline wrote.

12 The SQLite index: every table and its job

Eighteen tables/virtual tables. Grouped by job:

Curated-corpus relational core: `bills` (one row per curated bill; carries outcome, public_law, bipartisan_support, cluster, curator_note, outcome_events JSON), `sections` (167,742

rows; the parsed tree with `parent_section_id`, verbatim `text_self` and recursive `text_full`, canonical citation), `bill_versions`, `cosponsors`, `actions`, `subjects`, `citations` (reserved), `corpus_meta`, `source_freshness`.

Votes: `votes` (347 rows; per roll call: chamber, roll number, question, result, vote type, totals, per-party splits, bipartisan metric, source URL) and `vote_positions` (20,505 rows; member, party, state, position — passage-class votes only, which is what makes “how did the California delegation vote on CHIPS?” a one-join query).

Universe: `universe_bills` (46,168 rows; the status record), `universe_votes` (2,982 rows; party splits, no members), `universe_summaries` (28,786 rows; latest CRS summary), `bill_aliases` (97,401 rows; `alias_norm` → `bill_id`, source-tagged).

Search: `bills_fts`, `sections_fts` (corpus FTS; topic carried UNINDEXED so query-time filtering is free), `universe_fts` (title + all aliases + CRS summary — this is what makes the whole universe topically searchable), `section_embeddings` (the dense leg; partial coverage, reported honestly).

The universe row, quoted from `index/schema.py`:

```

1 CREATE TABLE IF NOT EXISTS universe_bills (
2     bill_id            TEXT PRIMARY KEY,
3     congress          INTEGER NOT NULL,
4     bill_type         TEXT NOT NULL,
5     bill_number       INTEGER NOT NULL,
6     title             TEXT,
7     introduced_date   TEXT,
8     origin_chamber    TEXT,
9     policy_area       TEXT,
10    sponsor_name      TEXT,
11    sponsor_party     TEXT,
12    sponsor_state     TEXT,
13    sponsor_bioguide  TEXT,
14    cosponsors_d     INTEGER NOT NULL DEFAULT 0,
15    cosponsors_r     INTEGER NOT NULL DEFAULT 0,
16    cosponsors_i     INTEGER NOT NULL DEFAULT 0,
17    cosponsors_total INTEGER NOT NULL DEFAULT 0,
18    latest_action_date TEXT,
19    latest_action_text TEXT,
20    outcome           TEXT,
21    public_law        TEXT,
22    outcome_events     TEXT,      -- JSON list, capped
23    vote_refs         TEXT,      -- JSON list of recorded-vote pointers
24    bipartisan_support REAL,
25    bipartisan_label   TEXT,
26    in_corpus         INTEGER NOT NULL DEFAULT 0
27 );

```

Note what is *not* here: no text, no sections. A 46k-row table of this shape is a few tens of megabytes and every query against it is milliseconds.

13 The Kùzu graph: nodes, edges, and what is actually populated

The graph implements the curated corpora’s structural layer. Node tables actually populated today: `Bill` (232; now carrying `outcome`, `public_law`, `bipartisan_support`, `cluster`), `BillVersion` (230), `Section` (167,742), `Sponsor` (747), `StatuteSection` (3,188 lazily-merged US Code targets), `DefinedTerm` (3,749), `AmendmentOperation` (6,875), `RollCallVote` (347), plus provenance plumbing. Edge tables populated: `HAS_VERSION`, `HAS_SECTION`, `PARENT_OF` (166,736 — the section tree), `SPONSORED_BY`, `COSPONSORED_BY`, `CITES_EXTERNAL` (9,412), `DEFINES` (3,689), `BY_REFERENCE` (387), `AMENDS` (6,612), `TARGETS` (6,842), `HAS_ROLLCALL` (347).

Declared but still empty (deliberately, with the gaps listed in `corpus_coverage`): `CITES_INTERNAL` (needs USLM `<ref>` parsing), `RESOLVED_TO/UnresolvedTermUse` (definition use-site resolution), `REFERRED_TO` committees.

14 Killing the 3.5-hour rebuild: the incremental path

The full graph build takes ~ 3.5 h on the Mac, dominated by parsing 167k sections through the citation/definition/amendment extractors. The June research pass established this is self-imposed, not a Kùzu limitation: since v0.4, Kùzu supports `COPY FROM` into *non-empty* tables, the documented fast path for bulk appends. The target maintenance discipline:

1. New bills $\rightarrow$ stage their node/edge rows as Parquet/CSV deltas $\rightarrow$ `COPY FROM` into existing tables. Seconds, not hours.
2. Property drift on existing nodes (an outcome changes when a pending bill passes) $\rightarrow$ small Cypher `SET` transactions. We already run this pattern: a drift-check script compares SQLite `bills.outcome` against graph properties and patches differences — the date-fix round found zero drifted corpus bills, verified in seconds rather than re-parsing for hours.
3. Full rebuilds only for schema changes.

The universe deliberately lives in SQLite only for now: every universe primitive is a filtered scan or aggregate (§2), so graph residency would add build cost without adding query power. Universe Bill nodes go into the graph (via `COPY FROM`) when there is a traversal that needs them — e.g., same-subject edges across the full universe.

Part IV

The evolution: how we got here

15 The chain at a glance

Step	When	What it added
v0.1 schema	spring	23-type design doc: bitemporal versioning, local definitions, reified amendments, derivation-typed edges
PR1	spring	bibliographic spine in Kùzu (Bill/Version/Section/Sponsor)
PR2	spring	CITES_EXTERNAL to US Code, statute targets lazily merged
PR3	May	DefinedTerm + DEFINES + BY_REFERENCE; aggregate definition tools
PR4	May	reified AmendmentOperation + AMENDS/TARGETS
Topics (P1–P3)	May	topic column end-to-end; redistricting seed (12 bills)
Round 1	Jun 9–10	secret_congress topic (29 bills, 6 clusters); billstatus.json enrichment for all bills; votes + outcomes + facet coverage; count_bills, get_bill_votes, find_bills_by_outcome
Round 2	Jun 10–12	the universe layer (46,168 bills); alias resolution (97,401 names); one-call bill cards; CRS summaries (28,876); universe-wide FTS; landing-page truthfulness

16 Why each step happened (the motivating failures)

v0.1’s bet: definitions are local. The original schema’s deepest insight, worth restating because everything since builds on its style: there is no global “AI system” node, because two bills’ identical surface forms carry different meanings. Definitions are scoped to their defining section; cross-bill questions are answered by *aggregating scoped facts*, never by merging them. Every later layer copies this pattern — curated cluster tags sit *beside* mechanical vote records, not instead of them; outcomes carry their evidence; coverage is per-facet, not a single number.

The eval drove the aggregates. The 20-question agent eval (Opus 4.7: 85%, Sonnet 4.6: 70%) failed systematically on one shape: set-valued and counting questions answered by paginated search + client-side counting. PR3/PR4 added `find_bills_defining`/`find_bills_amending` for the definitional/amendatory versions of that shape; Round 1’s `count_bills` closed the literal counting hole. The general law: *when the question shape is known, give the agent a primitive that returns the complete answer with a coverage note.*

The meeting notes drove passage dynamics. “Show negative bills that failed (why it fails?)”; “a lot of legislation is passed on a bipartisan vote in a single bill”; “how much it affected the vote.” These are not text questions, and the representation had no nouns for them — no outcome field, no votes, action trails on only ~30% of bills. Round 1 added the nouns mechanically (BILLSTATUS + roll-call XML are keyless), then curated 29 exemplars into the clusters the notes named (quiet bipartisan laws, bipartisan bills that died, omnibus vehicles, party-line contrasts).

Scale made the thesis testable, and the lack of scale was the critique. With only

29 curated exemplars, “bipartisan laws pass quietly” is an assertion. With the universe layer it is a query, and the answer is striking: of the 639 laws enacted by the 117th–118th, **361 (57%) passed with no roll call at all** (voice vote or unanimous consent — the quietest possible path), 244 passed bipartisan-to-near-unanimous, and only **13 (2%) were party-line** — exactly the reconciliation bills, CRA disapproval resolutions, and appropriations the thesis predicts. The curated clusters remain as *interpretation*; the universe is the *denominator*.

The research pass drove the interface restructure. The June review of deployed KG systems (GraphRAG, LightRAG, Saga, Wikidata/UMLS alias practice, the one published legislative KG, Kùzu’s own docs) returned one contradiction with our design: fine-grained primitives put graph traversal *inside* the LLM round-trip loop (§4). The responses: alias-table name resolution (the literature’s unanimous first step), the one-call card (Saga’s entity-card pattern at our scale), free-text search over the universe (so topical discovery does not require knowing which curated topic to search), and the routing rules in the system prompt.

Part V

The query surface: all 18 tools

17 The tool table

Grouped by the question shape they serve. “Layer” says which storage answers it.

Tool	Layer	Job (and when the agent should reach for it)
<i>Entry points</i>		
lookup_bill	SQLite	name/nickname/id → canonical bill_id(s); ALWAYS first when the user names a bill
get_bill_card	SQLite+Kùzu	one-call card: names, sponsor, outcome+evidence, votes, CRS summary, structure counts, in_corpus flag
search_corpus	SQLite	hybrid BM25+dense over curated full text, topic-scoped
corpus_coverage	SQLite	the coverage contract: topics, facets, universe split, known gaps
<i>Set-valued / counting (one call = complete answer)</i>		
find_universe_bills	SQLite	filters + free-text query over all 46k bills; the denominator tool
count_bills	SQLite	exact counts over curated bills, group-by topic/congress/outcome/...
find_bills_by_outcome	SQLite	curated bills by outcome / cluster / bipartisanship band
find_bills_defining	Kùzu	every bill defining a term (+ by-reference target filters)
find_bills_amending	Kùzu	per-bill rollout of who amends a USC section
find_definitions_of	Kùzu	every bill’s verbatim definition of a term, side by side
<i>Single-entity drill-down (curated bills only)</i>		
get_bill	SQLite	metadata + section table of contents (no bodies)
get_section	SQLite	verbatim text + canonical citation
get_bill_votes	SQLite	full vote record + outcome + events for one bill
get_defined_terms	Kùzu	every term one bill defines
get_amendments	Kùzu	every amendment operation one bill issues
get_amendments_targeting	Kùzu	operation-level detail on a USC section
get_citation_graph	Kùzu	typed citation neighborhood of a section

18 Routing: how the agent picks its first tool

The system prompt encodes a decision tree, because the wrong first tool costs round trips that no database speed can recover:

- User **names a bill** (“the CHIPS Act,” “H.R. 4346”) → `lookup_bill`, then `get_bill_card` on the resolved id. The card usually ends the question in two calls and reports whether full-text tools will work before any are tried.
- User asks a **topical/text** question → `search_corpus` on the right topic for deep passages, or `find_universe_bills(query=...)` for breadth across all 46k bills (titles + aliases + CRS summaries).
- User asks a **set or count** question → the aggregate tools, never paginated search.
- Ambiguity from `lookup_bill` is surfaced, not swallowed; `in_corpus=False` cards forbid text drill-down on that bill.

Part VI

The code, walked

19 Primer: the two query languages

SQLite FTS5. `CREATE VIRTUAL TABLE x USING fts5(col, ..., tokenize='porter unicode61')` builds the inverted index; `WHERE x MATCH 'fentanyl scheduling'` returns rows containing both stems; `ORDER BY rank` sorts by BM25. A column declared `UNINDEXED` is stored but not searchable — we use that for `bill_id/topic` so they can be filtered/joined without polluting relevance.

Kùzu Cypher. `MATCH (b:Bill {bill_id: $bid})-[:HAS_ROLLCALL]->(v:RollCallVote)` `RETURN v` binds node patterns by label and property, follows typed edges, returns rows. `STARTS WITH` on a primary-key-ish string is how we scope section-keyed rows to a bill. Parameters (`$bid`) are passed as a dict — never interpolate strings into Cypher.

20 Outcome derivation (ingest/billstatus.py)

The probes are compiled regexes over action text:

```
1 _PASSED_HOUSE = re.compile(r"passed[ /]agreed to in house|^passed house|on
   ↪ passage passed", re.IGNORECASE)
2 _PASSED_SENATE = re.compile(r"passed[ /]agreed to in senate|^passed senate",
   ↪ re.IGNORECASE)
3 _FAILED_PASSAGE = re.compile(r"failed of passage|on passage failed|failed
   ↪ passage", re.IGNORECASE)
4 _FAILED_CLOTURE = re.compile(r"cloture(?: on the )?.*not invoked|cloture
   ↪ motion .* rejected", re.IGNORECASE)
5 _VETOED = re.compile(r"vetoed by president", re.IGNORECASE)
6 _REPORTED = re.compile(r"reported (?:to|by)|placed on .* calendar", re.
   ↪ IGNORECASE)
```

Each match appends a dated, deduplicated event (`BILLSTATUS` repeats the same event from multiple source systems; we keep one per (type, day)). Roll-call results add failure evidence that action strings sometimes lack. Then the date-aware precedence:

```

1  # A failed floor vote is terminal only if nothing passed AFTER it.
2  def _last_date(*names: str) -> str:
3      return max((e["date"] or "" for e in events if e["event"] in names),
4                  ↪ default="")
5
6  last_pass = _last_date("passed_house", "passed_senate")
7  failed_passage_final = failed_passage and _last_date("failed_passage") >=
8      ↪ last_pass
9  failed_cloture_final = failed_cloture and _last_date("failed_cloture") >=
10     ↪ last_pass
11
12 if laws:
13     outcome = "enacted"
14 elif vetoed:
15     outcome = "vetoed"
16 elif failed_passage_final and congress_closed:
17     outcome = "failed_passage"
18 elif failed_cloture_final and congress_closed:
19     outcome = "failed_cloture"
20 elif passed_house and passed_senate:
21     outcome = "passed_both" if not congress_closed else "
22     ↪ died_after_passing_both"
23 ...

```

Line by line: `laws` (the `BILLSTATUS <laws>` element) trumps everything — enactment is the one fact the government states directly. A veto outranks floor failures. A failure is “final” only if no chamber passage postdates it (the H.R. 9495 lesson, §5). Chamber-passage flags then split by whether the Congress has adjourned: the same facts mean `passed_house_only` (alive) or `died_after_passing_house` (dead) depending on `congress_closed` — a frozen set of past Congresses that must be updated when the 119th ends (this is the one piece of calendar state in the file).

21 The alias contract (index/build_universe_tables.py + api/_impl.py)

Build-time normalization — the contract both sides must share:

```

1  def normalize_alias(s: str) -> str:
2      s = s.lower()
3      s = _PUNCT.sub(" ", s)      # punctuation -> spaces
4      s = _WS.sub(" ", s).strip() # collapse whitespace
5      if s.startswith("the "):
6          s = s[4:]
7      s = _YEAR_SUFFIX.sub("", s)  # drop trailing 'of 2022'
8      return s

```

Worked example: "The CHIPS and Science Act of 2022" → lowercase → punctuation/whitespace pass → "the chips and science act of 2022" → drop leading article → "chips and science act of 2022" → drop year suffix → "chips and science act". The same function runs at query time (imported from the same module — one definition, two call sites), so “the Chips & Science Act” from a user normalizes to an identical key. The year-drop is what merges “CHIPS Act of 2022” with “CHIPS Act”; the table keeps the original surface form so results can show what actually matched.

Query-time ladder in `lookup_bill`: (1) bill-id forms via the two ID regexes; (2) exact `alias_norm` match (the dictionary step that resolves the majority of mentions); (3) FTS5 over title+aliases(+summaries) as the fuzzy fallback. Each rung is tried only if the previous returned nothing; measured cost is 0.1–20 ms.

22 The bill-level bipartisan number (index/build.py)

```

1  def _final_passage_bipartisan(bs: dict | None) -> float | None:
2      """bills.bipartisan_support = the metric on the LAST passage-class

```

```

3     roll call (chronologically). Vehicle bills carry early votes from a
4     prior life under another name; the final vote is the one that
5     reflects the enacted coalition."""
6     if not bs:
7         return None
8     passage = [v for v in bs.get("votes", [])
9                 if v.get("vote_type") == "passage"
10                and v.get("bipartisan_support") is not None]
11     if not passage:
12         return None
13     passage.sort(key=lambda v: v.get("date") or "")
14     return passage[-1]["bipartisan_support"]

```

Three deliberate exclusions, each an honesty decision: amendment and procedural votes never represent the coalition for the bill (motions to recommit are designed to split parties); votes without party decomposition (voice votes have no roll at all) yield `None`, not 0 — `None` means “no partisan-decomposable evidence,” which is itself informative (57% of enacted laws); and `None` rows are excluded by band filters so “`bipartisan_support` ≥ 0.5 ” never silently includes unknowns.

23 The card assembly (`api/_impl.py::get_bill_card`)

The card is assembled at read time from five cheap queries (universe row, corpus row, aliases, votes, CRS summary) plus two optional graph counts:

```

1 # Graph section ids are '{urn_bill_id}::{xml_id}', so a prefix
2 # match scopes DefinedTerm / AmendmentOperation rows to this
3 # bill. Failures degrade to None (= unknown), never to 0.
4 urn_prefix = _to_urn_bill_id(bill_id) + "::"
5 for query, which in (
6     ("MATCH (d:DefinedTerm) WHERE d.defined_section_id STARTS WITH $p
7      ⇨ RETURN COUNT(d)", "terms"),
8     ("MATCH (a:AmendmentOperation) WHERE a.source_section_id STARTS WITH $p
9      ⇨ RETURN COUNT(a)", "amendments"),
10 ):
11     ...

```

Two details worth internalizing. First, the `None`-vs-0 discipline again: if the graph is unavailable, counts are `None` (“unknown”), because returning 0 would teach the agent the bill defines nothing. Second, why *compute-on-read* rather than precomputing card blobs (as Saga does): every component query is single-digit milliseconds at our scale, so precomputation would add a staleness liability and buy no agent-visible latency — the round trips were the cost, and one call is one call either way. If per-card cost ever grows (e.g., neighbor summaries), the switch to a precomputed `bill_cards` table is mechanical.

24 The universe build (`scripts/build_universe.py`)

The pipeline that makes 46k bills cost 19 seconds: GPO publishes one zip per (congress, bill type); the build streams each zip member through the same `parse_billstatus` used everywhere else, compacts to the record shape (cosponsor lists become party *counts*; events capped at 12; subjects at 12), skips the few auxiliary zip members with no bill identity (they would collapse into a garbage 117--0 row — found the hard way), sorts for deterministic diffs, and writes gzipped JSONL. Roll calls are fetched once per *unique vote URL* (2,982 across all three Congresses) with a disk cache, then joined back to every bill that references the URL. Summaries (BILLSUM) get the same treatment with a latest-version pick. Everything is re-runnable and idempotent; the zips are gitignored cache, the `jsonl.gz` artifacts are committed truth.

Part VII

Cheat sheet: your questions, answered

25 Index of questions

“You built this off an older branch?” No — verified zero commits behind `origin/master` at both build rounds; the branch is master + the two overnight commits. It *looked* old because rounds 1–2 were backend/data work with no visible pixels, and the landing page showed stale hardcoded counts — fixed by making the stats pipeline live (§7, landing section of the PR).

“If there’s 46,000 bills it needs to say 46,000.” Agreed and enforced: the landing fixture is regenerated from the index (46,168 tracked / 739 enacted / 3,329 roll calls / 232 full-text), and the numbers ship with a cache-bust version so deployed browsers cannot show stale ones. §10 for what each number means.

“Where are we pulling the bills from?” Five keyless primary sources, now also stated on the landing page: §8.

“Danny’s additional statistics — legislator notes / LA notes?” I read this as the CRS-written bill summaries in GovInfo’s `BILLSUM` collection (28,876 ingested; on every card and searchable: §8). If “LA notes” meant something else — e.g., committee-report text or CBO cost estimates — both exist as GovInfo collections and would follow the same ingest pattern; flag it and it’s a day’s work, not a redesign.

“How well the bill was passed?” That is `bipartisan_support` plus the per-vote party splits: §6. Margins, party splits, and the no-roll-call (voice/UC) case are all distinguished.

“Impact on sponsorship / donors?” Donor *behavior* data (FEC) is not ingested — it is a different identity universe (committees, filings) and a real research-design question, not an ingest script. The donor *topic* is represented (DISCLOSE Act, the platform-lobbying deaths of AICO/JCPA). §16.

“Can you get vote shares?” Electoral vote shares post-vote: out of scope tonight, agreed it’s the biggest ask. The path exists — member positions are keyed by bioguide ID, MIT Election Lab has results — but linking them honestly (redistricting, retirements, identification) needs a design discussion first.

“Why did bill X fail?” `get_bill_card` or `get_bill_votes`: outcome + dated evidence events (§5); for sets, `find_bills_by_outcome` / `find_universe_bills`.

“Was the CHIPS Act bipartisan?” Depends which vote — the worked example in §6 is exactly this bill, and the vehicle-bill trap (§9) is why the question needs the per-vote answer.

“What is ‘secret congress’ in our data?” A curated 29-bill topic (six clusters) *plus* a now-testable universe-level claim: 57% of 117th–118th laws passed with no roll call, 2% party-line. §16.

“Why is the Kuzu rebuild 3.5 hours and do we care?” Extractor parsing, not graph insertion; and we stop caring once appends go through `COPY FROM` deltas and property drift goes through Cypher SET: §14.

“What keeps the agent from making things up about coverage?” The facet-coverage contract in `corpus_coverage` plus prompts that treat `None` as “unknown” and status-only cards as text-free: §7, §23.

“What’s still missing?” Embeddings (handed off), CITES_INTERNAL and definition use-sites (declared, empty, listed in known gaps), committee/subject edges, universe nodes in the graph, measured media salience for the secret-congress clusters, FEC/electoral linkage. Each has a stated path; none blocks the current surface.